

태현 기반 통합 + 재성 확장성 반영 방안

PAC 2026 미션 1 · AHEAD 통합본 · 2026-10-10 · 기준 코드: 통합 저장소 main 85b13a6 · 근거: docs/donghan/overlap_review_20261009.md (4·5단계 같은 조건 실측 포함)

한 줄 요약

코드는 태현님 구조(RuntimeCore, LookaheadPolicy, pac_candidates, pac_robot_check)를 기반으로 합니다. 재성님이 정의한 현장·정보·비용·제약 변수는 모두 **설정 키**로 받습니다. 그래서 현장이 바뀌면 파일만 바꿉니다. 모든 확장은 "그면 지금 태현 결과와 같다"를 테스트로 보장하면서 하나씩 컵니다.

1. 왜 이 방식인가

- 4·5단계 실측(12 에피소드 짝비교): 팔레트 수는 차이가 없었습니다(태현 2.01, 재성 2.13, 차이 +0.12 [-0.16, +0.40]). 재성 쪽은 로봇 시간 -150 s, 찌그러짐이 약간 적고 맞물림이 2배였습니다. 태현 쪽은 결정이 3.4배 빠르고, 2 s 예산·실시간 실행기·테스트 33개를 갖추고 있습니다.
- 재성님 확장성의 핵심은 특정 알고리즘이 아니라 **현장 변수를 분리해서 근거와 함께 정하는 구조**입니다 (ALGORITHM_V3_3.md 11장, site_params/, 현장 프로파일 A~D).
- 태현 코드에는 이미 그 통로가 있습니다(environment.yaml → apply_environment). 다만 지금은 4·5단계 평가 스크립트에만 적용되고 ROS 런타임·3단계·V4.4 브리지에는 적용되지 않습니다. 이 통로를 넓히는 것이 출발점입니다.

2. 설정 구조: 재성님 4분류를 그대로 파일로

분류	파일	재성님이 정한 변수 (예)	정하는 법
현장 측정값	config/sites/<현장>.yaml (environment.yaml 확장)	컨베이어 속도 0.28 m/s, 정지 후 밀림 ≤0.03 m, 안정화 0.7 s, 팔레트 교체 시간, 로봇 사이클, 버퍼 칸 수·칸 크기, 팔레트 규격·높이·최대 하중	측정. 없으면 "가정값"으로 표시
운영 우선순위	같은 현장 파일 cost:	c_t, c_p(팔레트 1개 = 2000 / 1973 / 2368 s 중 하나로 통일), c_h(작업자 개입), c_u(미처리)	현장 의견 + 민감도 (site_oat)
물리 제약값	같은 현장 파일 constraints:	0.3 g 기동 검사, μ 0.4, 허용 하중 모델 (McKee × 습도 × 보관 × 0.8 ÷ 1.5), 무거운 박스 규칙	근거로 정함. 성능 때문에 바꾸지 않음
알고리즘 조정값	config/taehyeon/*.yaml (지금처럼)	탐색 깊이, 시나리오 수, 후보 수, θ_{buffer} - θ_{retrieve} , 비상 버퍼 칸, 마감 기준, 점수 가중치	검증 세트에서 sweep

- 재성님 현장 프로파일 A(기본)·B(수동 교체)·C(운송비 높음)·D(처리량 우선)는 config/sites/A.yaml ~ D.yaml로 옮깁니다.
- apply_environment 를 모든 실행 경로에 적용합니다: 4·5단계 평가, RuntimeConfig (3단계 Supervisor), ROS runtime_node, V4.4 브리지, 동한 pac_execution.
- default.yaml 과 environment.yaml 이 둘 다 "단일 원본"이라고 적힌 문제를 정리합니다. 팀 공통값은 default.yaml, 현장값은 현장 파일, 이렇게 하나씩만 둡니다. test_config_consistency.py 가 order JSON,

브리지 상수, 현장 파일까지 검사하게 합니다.

3. 항목별 반영 방법

3-1. 정보 모델 (재생: 계량 3 + 상류 2 + 미입고 수량, 대기 0)

현장 키	태현 코드에 반영
preview_weighed (기본 3)	탐색 창 안에서 무게까지 아는 박스 수
preview_upstream (기본 2)	종류만 아는 박스. 무게는 SKU 범위에서 시나리오마다 샘플
queue_capacity (기본 0)	물리적 대기 수. 미리보기 수와 분리(지금 태현 실시간 실행기는 $N+1=6$ 으로 묶여 있음)
order_list_known, horizon	미입고 수량 샘플은 horizon이 미리보기보다 길 때만 들어감. 지금 la_k3(horizon 5 = 3 + 2)에서는 0개라 두 키를 분리

- LookaheadPolicy 를 "결정적 1회"에서 "시나리오 k개, 평균 $(1-k)$ + 최악 k"로 확장합니다. **scenarios=1 0이면 지금 동작과 같습니다(하위 호환).**
- ROS 런타임: HighLevelDecider 가 지금은 미리보기 0개로 월드를 만듭니다. 컨베이어 관측 박스를 넘기도록 고칩니다.

3-2. 비용 단위 (재생: $J = c_t \cdot T + c_p \cdot N_{\text{pallet}} + c_h \cdot N_{\text{intervene}} + c_u \cdot N_{\text{unprocessed}}$)

- 태현 끝 상태 점수(팔레트 부피 비율)를 **J(초)**로 바꾸고 c_* , T_{change} 를 현장 파일에서 읽습니다. 환산은 사실상 같아서($0.0005/s \approx 2000 \text{ s/팔레트}$) 기본 결과는 거의 그대로입니다.
- 버퍼 넣기·꺼내기 비용을 시간 모델로 계산합니다. 재생님 θ_{buffer} 효과(실측 로봇 시간 -15%)가 이것으로 들어옵니다.
- **시간 모델은 하나로**(재생 §7-2b): 6단계 RobotFeasibility.cycle_time 을 Gazebo 실측으로 보정해서 씁니다. 고정 상수(놓기 8 s, 버퍼 4~5 s, 재적재 12 s)는 없앱니다. 현재 모델은 6.9 s, 재생 경로 합은 20.3 s, 상수는 8 s로 서로 다릅니다.
- 결과 보고는 재생님 원칙대로 T, N_{pallet} , 개입, 미처리를 **항상 따로** 같이 냅니다.

3-3. 제약 (하드마스크 선택형 모듈, pac_candidates)

키	선택지	비고
load_model	mckee_sf4 (태현 현행) / site_factor (재생 capacity.py: 판지 등급 × 습도 × 보관 × $0.8 \div 1.5$) / rated / no_stack	포장 종류(태현)와 함께 동작하게 합침
stack_lateral_g	0 = 끄 / 0.3 = 재생 기동 검사	stack_lateral_ok 를 프로토타입에서 옮기고 테스트 추가. 커먼 팔레트 약 +26%(실측 n=3)
heavy_on_light	끔 / share / per_box	코드 기본값과 YAML을 맞춤
mu	0.4 기본, 0.3-0.5 민감도	PyBullet(0.64/0.72)-Gazebo(0.6/0.9)와 통일

재적재 중에는 기동 검사가 적용되지 않습니다. 실측에서 검사를 켜도 통과 팔레트가 8/12였습니다. 재적재 이동에도 같은 모듈을 적용해야 합니다.

3-4. 자리 점수 (5-③)

- LayerPlacer 점수에 재성 항목(하중 여유 $\varphi(L)$, 맞물림, 비선형 안정성 $\varphi(S)$, 효율 7항목)을 **가중치 0** 기본으로 추가합니다. 실측에서 찌그리짐 -0.9개/에피소드, 맞물림 2배 효과가 있었습니다.
- 가중치는 YAML로 꺼냅니다. 지금 LayerConfig 가중치는 코드에만 있습니다.

3-5. 예외·운영 규칙

항목	모드
팔레트 마감	dead (태현 현행) / fill / J (재성: 교체 비용 vs 계속 비용)
재적재	no_place_only (현행) / opportunistic (재성: 같은 시나리오에서 이득 하한 > 0, 중간 상태 전부 검증)
컨베이어 정체	재성 7-2의 0~5단계(예방 → 일시 정지 → 빠른 모드 → 버퍼 → 교체·재적재 → 작업자)를 Supervisor 상태 전이로. queue_capacity, 정지 가능 여부, 정지 지연을 현장 키로
비상 버퍼	reserve_slots (재성 실험에서는 0 선택)
상태 확정	재성 6-1의 예정/확인 2단계, 계획 버전, 결정 마감 t_commit. 단일 StateManager에 추가

3-6. 검증 절차 (재성 9·11장을 공통 평가 스크립트로)

1. **하나씩 커서 효과 분리**: 기능마다 같은 데이터·같은 잡음·같은 시간 예산(2 s)으로 짝비교합니다(이번 h2h.py 를 확장).
2. 알고리즘 조정값은 **val에서만** 고르고 test-holdout은 마지막에 한 번만 씁니다. 선택 규칙: 완료율·제약 위반·마감 초과를 먼저 만족한 뒤 J 비교.
3. 현장 프로필 A~D에서 선택 결과가 어떻게 바뀌는지 봅니다(site_oat 방식).
4. **일반화**: 처음 보는 팔레트 규격, SKU 구성, 물림 순서, 잘못된 잔여 수량, 센서 오차에서 확인합니다.
5. **물리 판정 하나**: PyBullet 판정 도구에서 팔레트 크기 불일치를 고치고 μ 와 데크를 통일한 뒤 0.3 g 통과율을 다시 냅니다. 기존 74.9% 등은 시험 오류를 포함합니다.

4. 진행 순서

#	작업	끝난 기준	제안 담당
1	현장 파일 구조 + apply_environment 전 경로 적용	모든 경로가 같은 현장 값을 씀(일치 테스트)	태현 + 재성(값·근거)
2	J 단위 비용 + 단일 시간 모델 (cycle_time 보정)	기본 설정에서 기존 결과와 같음, Gazebo 실측과 오차 기록	태현 + 재성(실측)
3	정보 계층 + 시나리오 샘플	scenarios=1 에서 기존과 동일, 3+2 설정에서 효과 측정	태현
4	하드마스크 선택형 모듈(하중 모델, 기동 검사, μ)	모듈별 테스트, 끄면 기존과 동일	재성 → 태현 리뷰
5	자리 점수 재성 항목 추가	가중치 0에서 기존과 동일, 효과 측정	재성
6	예외 규칙 모드(마감 J, 기회형 재적재, 컨베이어 정지)	모드별 시나리오 테스트	재성 설계 · 태현 구현
7	공통 평가·물리 판정 스크립트	모든 비교를 이 스크립트로 냄	동한

공통 규칙: 모든 확장은 기본값을 "끔" 또는 "현행과 같은 값"으로 두고, "그면 지금 태현 결과와 같다"를 회귀 테스트로 묶습니다. 이렇게 하면 기반을 깨지 않고 재성님 변수를 하나도 빼지 않고 넣을 수 있습니다.

5. 먼저 정해야 할 것

1. **물리 제약 기본값:** 무거운 박스 규칙(끔/share/per_box), 0.3 g 기둥 검사(끔/끔), 허용 하중 모델. 팔레트 수가 약 30% 달라집니다.
2. **기준 현장 프로파일과 c_p 값:** 2000 / 1973 / 2368 s 중 하나로 통일합니다.
3. **기준 Gazebo 셀:** 재성 V4.4(받침대 0.40 m, 팔레트 1.1×1.1) / 태현 셀(0.50 m, 1.2×1.0). 버퍼 칸(2 vs 4)과 EOAT(Ø80 컵 vs 0.34×0.26 m)가 함께 정해집니다.
4. **미병합 브랜치와의 관계:** `claude/pensive-hypatia-9d5i3j` 가 단계별 단일화를 일부 다른 방향으로 이미 진행했습니다. `StateManager`를 `pac_common` 으로 합쳤고, 동한 `pac_execution` 을 삭제했습니다. 이 방안과 합칠지 먼저 정해야 합니다.